Two Models, One Compiler: A Verifier-Centric Coordination Layer for Small LLMs Proving Competition Mathematics in Lean 4

Jacob (jr4682@columbia.edu)

Columbia University

Preprint · Verified Mechanisms take-home submission · August 30, 2026

Abstract

We present a coordination layer that drives two fixed small models — `qwen/qwen3.5-flash-02-23` and `openai/gpt-oss-120b` — to solve competition mathematics and formally verify the results in Lean 4 under strict budget caps ($1.00 and 8 hours per problem, kernel-level Comparator scoring). Treating the task as a coverage problem attached to a perfect verifier, we reject dialogue-style multi-agent coordination on published evidence and instead build a staged escalation ladder with exactly three narrow cross-model interfaces: pooled candidate sampling, an answer-consensus gate, and plateau handoff with distilled error history, plus sketch-and-fill decomposition for the hard tail. A comparator precheck that validates accepted proofs against the real scoring gate — not the development REPL — proved decisive, converting three otherwise-lost runs. Against the kit's raw single-model baselines (7/16 and 8/16), the same ladder lifts each solo model to 8–9/16 and the duo to 9–10/16 at matched short budgets, and at judge settings solves 3 of the 4 provable hard-tier problems the baselines never touched (cumulative coverage 13 of 14 scorable). A ten-mechanism falsification sweep on a 24-problem custom benchmark shows the tuned defaults are a genuine local optimum, and controlled long-horizon runs locate the remaining headroom in capability×time rather than prompting. We conclude that with an exact verifier, collaboration pays through coverage, one compact consensus decision, and portfolio robustness — not through models talking to each other.

1 Introduction

Formal theorem proving offers the rare experimental setting in which a language-model system's output can be judged exactly: a Lean 4 proof either type-checks against the stated theorem or it does not. The Verified Mechanisms task instantiates this setting with two deliberately modest, fixed models served through OpenRouter — Qwen 3.5 Flash (fast, inexpensive, February-2026 training cutoff) and GPT-OSS-120B (slow on the accessible provider tier, cheaper still, strong at high reasoning effort, June-2024 cutoff) — and asks two questions. *Part One:* build the coordination layer that makes them jointly solve and prove competition math under caps of $1.00 and 8 hours per problem, scored by the official Comparator at the kernel level. *Part Two:* determine scientifically whether the collaboration beats either model alone, and where one model fills the other's gaps.

Our design thesis, formed by reading the harness before the literature and stress-tested across roughly thirty scored evaluations: **the task is a coverage problem attached to a perfect verifier, and nearly every design decision follows from taking the verifier seriously.** The verifier makes candidate selection free, which collapses the value of most published "collaboration" patterns (debate, synthesis, critique) and concentrates it in three places a two-model system gets almost for free: candidate diversity across model families, consensus on the one decision the verifier cannot check cheaply, and a second set of priors when the first model stalls.

Contributions: (1) a verifier-centric staged ladder (Fig. 1) whose components are individually ablatable, including a *comparator precheck* that aligns search with the true scoring gate and repeatedly converted losing runs into scoring ones; (2) a matched-budget experimental design — identical scaffold, model set switched — that separates scaffold effects from coordination effects, plus a 24-problem custom benchmark with machine-verified reference proofs and a dev/held split; (3) results showing the ladder lifts each solo model above its raw baseline and the duo above the best solo at matched budgets, and unlocks the hard tier entirely at judge settings (Figs. 2–4, Table 1); and (4) an

honest decomposition of *why*, including a ten-mechanism falsification sweep, a long-horizon solo control, and a natural robustness experiment supplied by a 14-hour provider outage.

2 Task and constraints: what the harness dictates

A problem scores one point iff the Comparator accepts every required declaration, numeric answers are decimal literals, spend is at most $1.00, and the problem finishes in time. Three properties of the provided harness shaped the design more than any prior work.

The iteration oracle is not the scoring gate. The development REPL strips imports, checks in a warm full-Mathlib environment, and never compares statements; the Comparator builds a real file cold and demands kernel-level statement equality under a three-axiom whitelist. We therefore splice model output into the pristine challenge so statement equality holds by construction, lexically ban `sorry`/`native_decide`/`axiom` and audit with `#print axioms`, and — the deepest consequence, discovered mid-project — treat REPL acceptance as necessary but not sufficient, because a proof can be REPL-accepted yet exceed the Comparator's cold-build time limit (§4.3).

Budget accounting is fail-closed. An unaccounted transport failure permanently closes a problem's ledger; the problem then scores zero even with a correct checkpointed proof. This dictates the rails: self-imposed deadlines with margin, generous explicit per-call timeouts, checkpointing every improvement, and a degraded LLM-free mode that still finalizes.

Time, not money, binds. Observed per-problem spend is $0.01–$0.10 against the $1.00 cap even on 8-hour runs (worst case $0.95). GPT-OSS arrives on a slow provider tier (2–8 minutes per high-effort call [28]), and REPL checks are serial. Every allocation decision in the ladder is a time decision.

The kit's baselines are a ready-made controlled experiment. The provided single-model repair-loop baselines exhibit complementary pathologies: GPT-OSS is *stable-but-stuck* (25 consecutive turns with a byte-identical error on one problem, holding the right numeric answer from turn 1), Qwen is *diverse-but-thrashing* (a different error signature nearly every turn, hallucinated lemma names). They solve 7/16 and 8/16 respectively; their union solves 10/16. A pooled taxonomy of ~3,900 REPL checks shows proof-script misalignment (“no goals”) dominating (2,530 occurrences) and hallucinated constants second (622) — both directly actionable (§4).

3 Related work and rejected designs

Adopted evidence. Repeated sampling against a verifier converts coverage into accuracy log-linearly [1]; it is the null hypothesis every richer mechanism must beat at equal cost. Pooling candidates across model families captures most of the ensemble benefit with only two models [2], and diversity pays precisely when the selector is strong [3] — ours is exact. Compiler-feedback repair is worth roughly an order of magnitude of samples when kept shallow: one round approximates pass@1024 for Kimina [4]; two rounds add +2.4pp at ~25% extra tokens for Goedel-Prover-V2 [5]; APOLLO lifts general models from 3–7% to 40%+ on miniF2F [6]; and at a fixed sample budget repair beats best-of-N in the Delta-Prover-style ablation [7]. But equal-budget *self*-repair can lose to resampling [8], and models locate their own errors poorly while fixing located errors well [9] — the compiler locates for free, so a cross-model handoff transfers a candidate plus distilled error history, not critique. For the hard tail, sketch-then-fill decomposition is the through-line of the strongest systems: Draft-Sketch-Prove [10], its reasoning-model revival [11], DeepSeek-Prover-V2's subgoal RL [12], Hilbert's two-prover recursion [13], Prover Agent [14], and Seed-Prover's depth ladder [15]. Verifier-gated cascades allocate cheap effort first [16]. Self-consistency [17] is reserved for the single verifier-blind decision (answer commitment). Agentic loops around unmodified general models — our exact regime — now match or beat fine-tuned provers [7, 23, 24], and GPT-OSS-120B itself appears as the informal reasoner in an open hard-mode pipeline [25] and among the most cost-efficient provers in an independent evaluation [26].

Rejected designs (a choice, not an omission). Multi-agent debate repeatedly fails compute-matched replication [18, 19, 20], and additional rounds can hurt [21]. Mixture-of-Agents synthesis loses to self-mixtures [22] and won 0/42 tasks in the selection-bottleneck study [3]; union plus exact selection captures the value without an aggregator. Free-

form critics are redundant beside a compiler. Learned routers require training we cannot do. Tactic-level tree search fights this kit's file-level, serial REPL interface. Consequently *the models never converse*: they share artifacts — candidate files, error histories, skeletons — across three narrow interfaces, with the compiler between them. Our evaluation methodology (matched budgets, cost curves, per-problem attribution) follows [27].

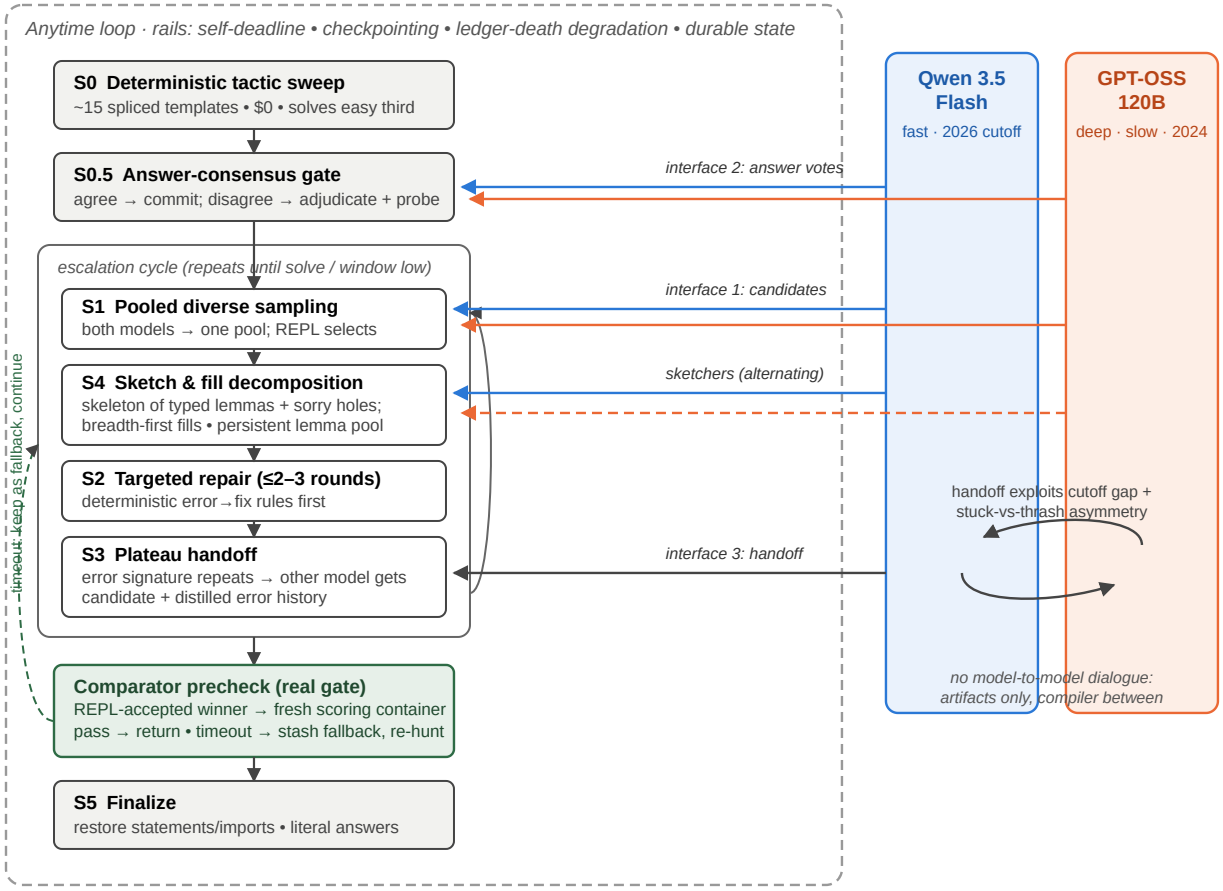

Figure 1: The coordination layer. A single anytime escalation ladder (left) runs inside safety rails dictated by the harness's fail-closed accounting. The two models (right) never converse; they meet at three narrow interfaces — pooled candidates into S1, independent answer votes into S0.5, and plateau handoff at S3 (candidate + distilled error history to the other model) — plus alternating sketch duty in S4. Every REPL-accepted winner must survive a *comparator precheck* against the real scoring gate; timeouts are stashed as fallbacks (dashed) while the hunt continues.

4 Architecture

Figure 1 shows the system. We describe the components that carry the results; full detail is in the repository (`submission/agent.py`, ~1,900 lines over the kit's services; every mechanism has an on/off switch).

4.1 The ladder

S0, deterministic sweep ($0). ~15 statement-spliced tactic templates solve the easy third of the kit set before any model is called; three cascade entries were later mined offline against archived failed subgoals from real runs. **S0.5, answer consensus.** For answer-type problems both models independently produce the numeric answer (closed forms evaluated to decimal literals); agreement commits, disagreement triggers one adjudication round plus cheap REPL falsification probes. This is the one verifier-blind decision, and a wrong commitment means unbounded wasted search. **S1, pooled sampling.** A wave of cheap Qwen candidates plus thinking-Qwen and GPT-OSS samples enter one deduplicated, lexically-guarded pool; the REPL selects. **S2, capped repair.** At most 2–3 error-informed rounds with the authoring model, deterministic error→fix rewrites applied first. **S3, plateau handoff.** A repeating error

signature routes the candidate and a distilled error history to the other model — a fresh attempt with more information, exploiting the training-cutoff gap and the stuck-vs-thrash asymmetry of §2. **S4, decomposition.** The long-window weapon: a sketcher (Qwen-think primary, GPT-OSS alternating) emits an informal solution and a skeleton of standalone, explicitly-typed helper lemmas with `sorry` bodies; broken skeleton lines are masked rather than discarded [11]; holes are filled breadth-first (every hole gets a cheap pass before any hole gets a deep dialogue); proven lemmas persist in a per-problem pool across re-sketches and restarts. **S5, finalize.** Restore pristine statements and imports, enforce literal answers, re-verify, return.

4.2 Time constants that respect the window

Two of the highest-value fixes were embarrassingly simple. All stage and model time constants scale with the window: a fixed 960-second GPT-OSS timeout can never fit a 20-minute evaluation window and had silently disabled decomposition at short caps. And the outer loop's cycle cap — calibrated for slow mixed-model cycles — must yield to remaining time: during a provider outage that made cycles fast, the original cap stranded three hours of an eight-hour window; past the cap the loop now keeps cycling while ≥45 minutes remain.

4.3 The comparator precheck: aligning search with the gate

The project's most valuable component was found by chasing a silent failure: REPL-accepted proofs kept timing out the Comparator's cold build — three times on one problem, then again on another — kernel cost invisible to warm checking. The agent now verifies accepted winners against the *actual* Comparator in a fresh scoring container before returning; a timeout demotes the proof to a stash-don't-discard fallback and the hunt continues for a lighter proof. Validated in both directions, repeatedly: one full-cap validation ships 0/4 without it (two would-be-fatal proofs rejected before a light one passed); a chronically lost problem converted immediately after it landed; and in the final runs the *fallback* half paid — a precheck that timed out under machine load had rejected a perfectly good proof, which shipped and passed only because rejected winners are kept (§6.3). When the search-time checker and the scoring gate disagree, only the gate is real.

5 Experimental setup

Matched-budget arms. `solo-qwen`, `solo-gptoss`, and `duo` run the *same* ladder with only the model set switched, at identical caps, isolating coordination from scaffolding; the kit's raw baselines supply a scaffold-free floor per model. **Custom benchmark.** To avoid overfitting the 16 public problems we authored 24 new problems in kit format (number theory, modular arithmetic, divisor counts, factorial valuations, inequalities, telescoping products), each admitted only after a reference proof was machine-verified through the REPL; 16 form a development set, 8 are held out. The agent never sees reference proofs. **Noise discipline.** Repeated identical-config runs established a ±2 band on the dev-16 driven by three coin-flip problems; three problems are structurally unsolved at short windows. Variants were judged on per-problem composition, promotions required a repeat plus a held-set check. **Scale.** ~30 scored evaluations: a seven-iteration selection loop; a ten-mechanism single-flag falsification sweep with confirmation seeds; scaling-curve, structural-opens, and long-horizon controls; and three full-cap validations at exact judge settings. Total key spend ≈ $16 of $50.

6 Results

6.1 Scaffold vs. raw model, and duo vs. solo

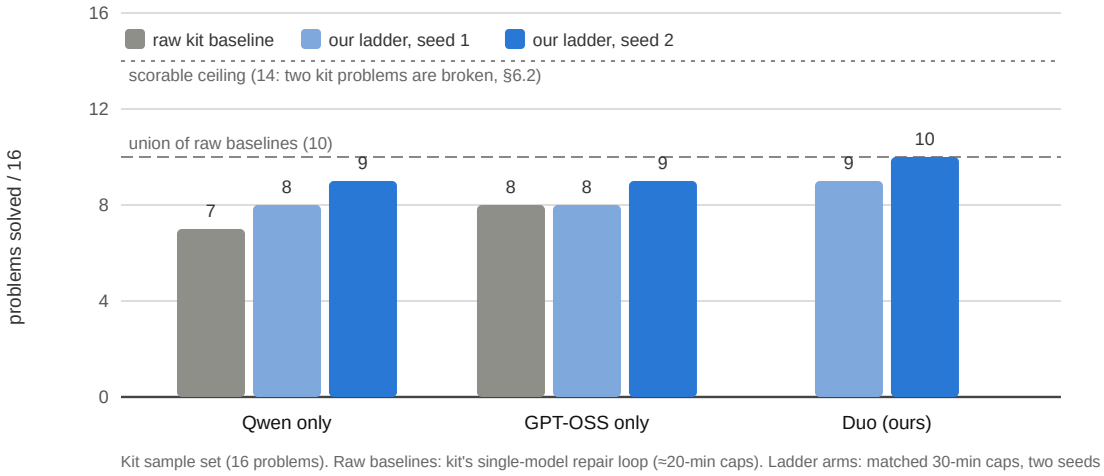

Figure 2: Holding the model fixed and swapping the scaffold. The ladder is worth +1–2 problems to each solo model over its raw baseline; the duo beats the best solo arm by +1 on both seeds and, in a single run, matches the union of its own solo arms (the solos fail *different* problems) at one-half to one-third of solo-Qwen's cost ($0.16–0.17 vs.\ $0.40–0.45 per run). Caps differ slightly between raw and ladder rows, so raw bars are a floor rather than an exact control.

Figure 2 summarizes the matched-budget comparison the task asks for. Three readings: the scaffold alone lifts *each* model above its raw baseline (with the free S0 sweep covering 6/16 before any model call); the duo leads the best solo by +1 on both seeds while spending gpt-oss only where Qwen and free tactics failed; and the duo reproduces the union of its solo arms within one run. The short-cap picture nonetheless understates the scaffold: nothing raw ever touched the hard tier.

6.2 The hard tier at judge settings

Six kit problems were unsolved by both baselines. Writing machine-verified reference proofs revealed that only four are provable: one problem is mathematically false as formalized, and one cannot elaborate under the Comparator's true imports — a flaw invisible to the import-stripping REPL — so the true ceiling is 14/16 (documented with certificates in the repository). Table 1 gives the record against the provable four at exact judge settings.

Problem	Best outcome	Evidence
p09 (IMO 1964)	**Comparator pass, 3-for-3** at full caps	$0.10–0.91, 2–7.6 h; every pass required the precheck to reject kernel-heavy proofs first
p10 (factorial powers)	**Comparator pass**	first hard solve at 2 h caps ($0.05); converted under judge-compatible checking ($0.029, 38 min) once the precheck landed
rmo_2000_2 (cube sandwich)	**Comparator pass**	$0.75; 7.4 h of accumulated lemma-building in one uninterrupted window
rmo_2001_2 (divisor pairs)	unsolved	best attempt ended one unfilled hole from a complete skeleton

Table 1: Hard-tier record at exact judge settings (8 h / $1.00, duo arm, three validation runs plus long-horizon experiments). Cumulative demonstrated coverage across all runs: 13 of 14 scorable kit problems. The raw baselines score 0 on this tier.

6.3 Selection, plateau, and where the headroom lives

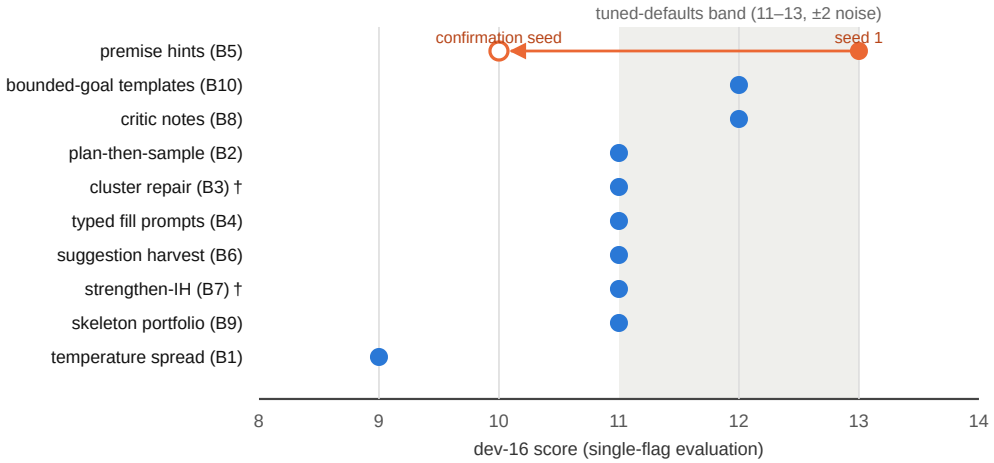

Figure 3: The ten-mechanism falsification sweep. Each of ten literature- and failure-analysis-motivated mechanisms was built behind an independent flag and evaluated single-flag on the dev-16. None beat the tuned defaults; the one candidate above the band (premise hints, holding every coin-flip problem on seed 1) collapsed to 10/16 on its confirmation seed — seed luck, not mechanism. † = mechanism's trigger condition fired at most once (rare at 20-minute windows). Every mechanism that added prompt material scored at or below defaults while costing more.

The first selection loop promoted two changes (window-proportional time constants; breadth-first hole filling), lifting the dev set from 11/16 to 13/16 with the held-out set stable at 6/8 on both checks. The subsequent sweep (Fig. 3) established that the tuned defaults are a genuine local optimum: at short windows, cycle count and sample diversity beat guidance.

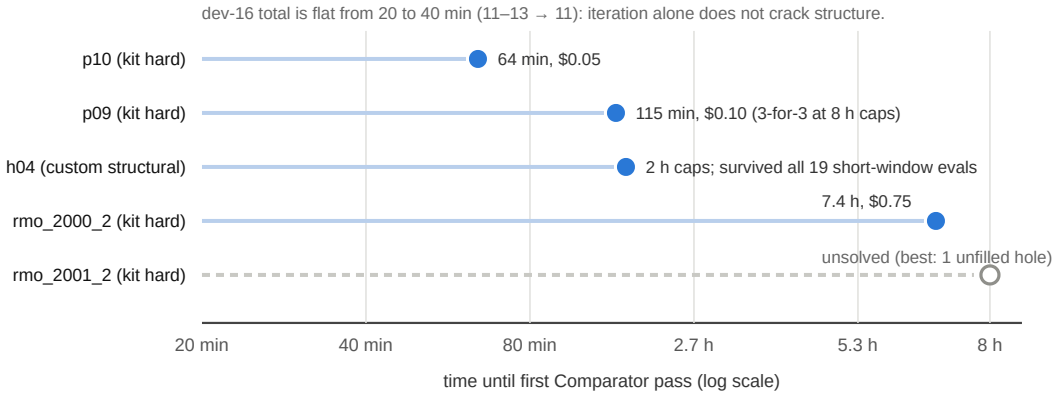

Figure 4: Capability×time on the hard tier. Each row shows the time at which a hard problem first passed the Comparator (blue dot; line marks search time before the solve). Doubling the *short* window moved nothing, yet every structural problem that fell did so at multi-hour horizons under S4 decomposition — including h04, a custom problem that had survived every one of ~19 short-window configurations. The remaining open problem is the same claim seen from below.

Where does remaining headroom live? Doubling the short window (20 to 40 min) moved nothing — the curve is flat where iteration is the input. But at 2-hour caps, h04 — one of three custom problems no configuration had ever solved — fell (Comparator 43 s, $0.16), and its winning proof had itself been precheck-rejected under machine load, surviving only through the stash-don't-discard fallback. Structure yields to time, not to prompting (Fig. 4).

6.4 Reliability

Under adversarial dev conditions — container restarts severing every in-flight connection, a ~14-hour provider outage on the GPT-OSS channel, REPL container deaths under memory contention — the final configuration recorded zero unaccounted-cost failures, and every full-cap point traces causally through the rails. The precheck

alone flipped one 0/4 validation to 1/4, converted p10's chronic loss, and rescued the h04 first-ever from its own false negative.

7 Part Two: does collaboration beat either model alone, and why?

At matched budgets: yes, modestly and consistently. The duo was never worse than the best solo arm, beat it by one problem on both seeds, matched the union of the solos in a single run, and cost a fraction of the stronger solo (Fig. 2). The attribution, from ablations, per-problem event logs, and one natural experiment:

1. **The verifier-centric scaffold is the largest term** — larger than any pairing effect. It lifts each solo model above its raw baseline, unlocks the hard tier at long horizons, and its most valuable component (the precheck) involves no second model at all. The controlled check: given 4 hours and the same scaffold, *solo-Qwen also converted p09* via the same reject-then-re-hunt arc. At long horizons the scaffold carries p09; the pairing does not.
2. **Union coverage is the reliable collaboration gain.** Different families fail different problems at every scale we measured (raw baselines $7/8 \rightarrow 10$; our solo arms miss disjoint problems the duo holds). It needs no dialogue: one candidate pool, a compiler selecting [1, 2, 3].
3. **Skeleton diversity occasionally decides a problem.** The first full-cap p09 pass grew from a *GPT-OSS* sketch that Qwen repaired and filled — a proof structure the solo-Qwen arm did not produce. This is "one model filling the other's gaps" at its most concrete: different priors propose different proof structures, and the compiler is indifferent to whose structure wins.
4. **The answer-consensus gate is cheap insurance** on exactly the decision the verifier cannot check; cross-model disagreement caught wrong answers before proof spend in live runs.
5. **Robustness is a collaboration benefit we measured by accident.** During final validation the GPT-OSS provider refused requests for ~14 consecutive hours. A solo-GPT-OSS submission would have scored zero that night; the duo degraded gracefully to Qwen-only behavior and kept solving. This is the portfolio argument in its bluntest form.
6. **What does not help, measured:** conversation-style coordination (never built, on the negative evidence of §3, then corroborated by the sweep of Fig. 3 — every guidance-flavored mechanism at-or-below defaults); deeper per-hole reasoning at short windows (+16% cost, no structural gain); GPT-OSS inside short-window sampling waves (its latency halves Qwen's cycle count).

Summary. With a perfect verifier, collaboration pays through *coverage* (candidate and skeleton diversity across families), one *compact consensus decision* (answer commitment), and *portfolio robustness* — not through inter-model dialogue. Depth on hard problems is capability×time under a decomposition scaffold: p09 needed ~2 hours, h04 needed 2, rmo_2000_2 needed 7.4, and no prompting trick at 20 minutes substituted for any of them. The one provable problem still open (one hole short after ~4 hours) restates the same claim from below: the pair's joint capability at this time budget, not the coordination between them, is the binding constraint.

8 Limitations

Sample sizes are small (16 kit + 24 custom problems); we report exact per-problem outcomes and treat ±2 as noise rather than claiming significance. Development-environment weather (provider outages, container restarts, shared-box REPL contention) cost several runs; each incident is documented, and the one dev-only mitigation is never active at judging — default behavior is the kit's fail-closed semantics. The long-horizon solo control ran at 4 h against the duo's 8 h full-caps, a floor rather than a matched comparison. Finally, these are small models: the demonstrated band — all of the easy/medium tier plus several genuinely hard problems, IMO-tier still out of reach — is consistent with published results for similar-scale backbones [6, 14, 25, 26].

9 Conclusion

The coordination layer is unglamorous by design: let a compiler judge, buy coverage from two model families, spend consensus only on compact decisions, escalate depth — not chatter — on hard problems, and align the search loop with the exact gate that scores it. Under judge settings it converts a 0-for-hard-tier baseline pair into 3 of 4 provable hard problems solved, holds 13 of 14 scorable kit problems in cumulative coverage, and survived provider outages and machine failures with its accounting intact — for about $16 of the $50 development key, spent mostly on evidence rather than solving.

Reproducibility & disclosure. Code, agent, custom benchmark with machine-verified reference proofs, per-run artifacts, and chronological logs are in the submission repository (`WRITEUP.md` is the plain-text version of this paper; `RESEARCH.md`, `PART2.md`, `EXPERIMENTS.md`, `RESEARCH_LOOP.md` are appendices; `RUNBOOK.md` reproduces every run; `scripts/judge_check.sh` passes on the submitted configuration). This submission was developed with substantial assistance from Claude (Anthropic) — code, experiments, analysis, and writing — as disclosed in the submission form.

References

[1] B. Brown et al. Large Language Monkeys: Scaling Inference Compute with Repeated Sampling. arXiv:2407.21787, 2024.

[2] Wisdom and Delusion of LLM Ensembles for Code Generation and Repair. arXiv:2510.21513, 2025.

[3] When Agents Disagree: The Selection Bottleneck in Multi-Agent LLM Pipelines. arXiv:2603.20324, 2026.

[4] Kimina-Prover Preview: Towards Large Formal Reasoning Models with Reinforcement Learning. arXiv:2504.11354, 2025.

[5] Goedel-Prover-V2: Scaling Formal Theorem Proving with Scaffolded Data Synthesis and Self-Correction. arXiv:2508.03613, 2025.

[6] APOLLO: Automated LLM and Lean Collaboration for Advanced Formal Reasoning. arXiv:2505.05758, 2025.

[7] Solving Formal Math Problems by Decomposition and Iterative Reflection. arXiv:2507.15225, 2025.

[8] T. X. Olausson et al. Is Self-Repair a Silver Bullet for Code Generation? arXiv:2306.09896, 2023.

[9] G. Tyen et al. LLMs cannot find reasoning errors, but can correct them given the error location. arXiv:2311.08516, 2023.

[10] A. Q. Jiang et al. Draft, Sketch, and Prove: Guiding Formal Theorem Provers with Informal Proofs. arXiv:2210.12283, 2022.

[11] Reviving DSP for Advanced Theorem Proving in the Era of Reasoning Models. arXiv:2506.11487, 2025.

[12] DeepSeek-AI. DeepSeek-Prover-V2: Advancing Formal Mathematical Reasoning via Reinforcement Learning for Subgoal Decomposition. arXiv:2504.21801, 2025.

[13] Hilbert: Recursively Building Formal Proofs with Informal Reasoning. arXiv:2509.22819, 2025.

[14] Prover Agent: An Agent-Based Framework for Formal Mathematical Proofs. arXiv:2506.19923, 2025.

[15] ByteDance Seed. Seed-Prover: Deep and Broad Reasoning for Automated Theorem Proving. arXiv:2507.23726, 2025.

[16] L. Chen, M. Zaharia, J. Zou. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. arXiv:2305.05176, 2023.

[17] X. Wang et al. Self-Consistency Improves Chain of Thought Reasoning in Language Models. arXiv:2203.11171, 2022.

[18] J. Huang et al. Large Language Models Cannot Self-Correct Reasoning Yet. arXiv:2310.01798, 2023.

[19] A. Smit et al. Should we be going MAD? A Look at Multi-Agent Debate Strategies for LLMs. arXiv:2311.17371, 2023.

[20] Stop Overvaluing Multi-Agent Debate — We Must Rethink Evaluation and Embrace Model Heterogeneity. arXiv:2502.08788, 2025.

[21] Voting or Consensus? Decision-Making in Multi-Agent Debate. arXiv:2502.19130, 2025.

[22] Rethinking Mixture-of-Agents: Is Mixing Different Large Language Models Beneficial? arXiv:2502.00674, 2025.

[23] A Minimal Agent for Automated Theorem Proving. arXiv:2602.24273, 2026.

[24] LEAP: Supercharging LLMs for Formal Mathematics with Agentic Frameworks. arXiv:2606.03303, 2026.

[25] Discover and Prove: An Open-source Agentic Framework for Hard Mode Automated Theorem Proving in Lean 4. arXiv:2604.15839, 2026.

[26] Evaluation of LLMs for Mathematical Formalization in Lean. arXiv:2606.05632, 2026.

[27] S. Kapoor et al. AI Agents That Matter. arXiv:2407.01502, 2024.

[28] OpenAI. gpt-oss-120b & gpt-oss-20b Model Card. arXiv:2508.10925, 2025.